SIT707 Software Quality and Testing

Week 1 – Evidence of Learning
Name: Ayush Indapure
Unit: SIT707
Topic: Software Quality and Testing Fundamentals

What is Software Quality

Software quality refers to how well a software system meets user requirements and performs reliably in real-world situations.

Key Aspects of Software Quality

- Reliability – software should work consistently without crashes
- Usability – users should be able to interact with the system easily
- Performance – the system should respond quickly and efficiently
- Security – user data must be protected

Example

For example, in a banking application, software quality ensures that transactions are processed correctly and that user data remains secure.

Example – From My Project *Streeeats*

While developing my web project **Streeeats**, I realised how important software quality is in real-world applications.

The platform allows users to discover underrated food stalls near their location. One challenge I faced was ensuring that **location-based filtering worked accurately**. If the system calculated distances incorrectly or failed to load results, users would not see nearby food stalls.

I also had to ensure that **Google Maps links generated correctly** for each food stall. A small mistake in the query string could take users to the wrong location.

These experiences helped me understand that even small issues in code can affect usability and reliability, which directly impacts overall software quality.

Examples

ChatGPT experienced a global outage on February 4, 2026, affecting users worldwide. The disruption was characterized by elevated error rates, timeouts, and inaccessible chat interfaces, with users encountering messages like "something seems to have gone wrong".

Why Software Quality Matters

High software quality is important because software systems are used in critical areas such as finance, healthcare, transportation, and communication. Poor quality software can lead to system crashes incorrect data processing financial losses loss of user trust

For instance, if a payment application fails during a transaction, it may result in duplicate payments or lost funds.

Ensuring software quality helps organizations deliver reliable and trustworthy products.

Faults, Errors and Failures

In software engineering, faults, errors, and failures are closely related concepts.

Fault
A fault is a mistake or defect in the code written by a developer.

Error
An error occurs when the software enters an incorrect internal state due to a fault.

Failure
A failure happens when the software behaves incorrectly or produces an unexpected result.

Example

If a developer writes an incorrect formula in a calculation module, that is a fault.
When the system processes data using that incorrect formula, it creates an error.
If the user receives an incorrect output, that becomes a failure.

Why Software Testing is Necessary

Software testing plays an important role in identifying faults before software is released to users.

The main purposes of testing include:

- detecting bugs early in development
- improving system reliability
- ensuring the software meets requirements
- reducing the cost of fixing problems later

Testing helps developers identify issues before they affect real users.

Without proper testing, even small coding mistakes can lead to major failures in production systems.

Testing in the Software Development Process

Common levels of testing include:

Unit Testing
Testing individual components or functions.

Integration Testing
Ensuring that different modules work correctly togeth

System Testing
Testing the entire application as a complete system.

Acceptance Testing
Verifying that the software meets user requirements.

Active Learning Session

I attended the weekly active learning workshop conducted by **Shruti on Tuesday at 4 PM**.

The session was engaeing and interactive, and I really enjoyed her teaching style. During the workshop, she discussed the importance of software testing in real-world systems.

She also shared insights from her **research on context-aware systems**, explaining how modern applications need to adapt to different user environments and conditions.

This discussion helped me understand that software testing must consider various contexts, such as user behavior, device types, and environmental factors.

Importance of Testing in Real World

This week's learning helped me understand how critical software testing is in real-world systems.

As a web developer, testing is especially important because modern applications rely on many components such as APIs, databases, and user interfaces.

If a web application is not tested properly, users may experience issues such as:

- broken user interface components
- failed API requests
- slow performance
- security vulnerabilities

Testing ensures that the application behaves correctly and provides a reliable experience to users.

Reflection of what I learnt this week

Key Takeaways

Interesting things I've learned as of now

From this week's learning, I understood several important concepts about software quality and testing.

Key takeaways include:

- Software quality ensures that systems meet user expectations and perform reliably.
- Faults in code can lead to errors and system failures if not detected early.
- Software testing is essential for identifying defects before deployment.
- Real-world failures highlight the importance of thorough testing.
- Developers must incorporate testing practices throughout the development lifecycle.

These concepts will be valuable when developing reliable software applications in real-world scenarios.